

CSE4261: Neural Network and Deep Learning

Assignment-13

Name: Md Sajjad Hossain

ID: 2011176125

Session: 2019-20

August 04, 2025

1 Synthetic Face Generation using Deep Convolutional GAN

1.1 Objective

The objective of this task was to generate realistic synthetic face images by training a Deep Convolutional Generative Adversarial Network (DCGAN) using the CelebA dataset.

1.2 Dataset

We used the CelebA dataset, which contains over 200,000 celebrity face images. For training, each image was resized to 64×64 pixels with three color channels. The dataset was normalized to the range $[-1, 1]$ to match the `tanh` activation output of the generator.

1.3 Model Architecture

Generator:

- Input: 100-dimensional noise vector.
- Four transposed convolutional layers with Batch Normalization and LeakyReLU activations.
- Final layer: 3-channel output using `tanh` activation to match the image pixel range.

Discriminator:

- Input: $64 \times 64 \times 3$ image.
- Four convolutional layers with Batch Normalization and LeakyReLU activations.
- Final Dense layer with `sigmoid` activation to produce a real/fake probability.

1.4 Training Details

- Epochs: 21
- Batch size: 128
- Optimizer: Adam with learning rate 2×10^{-4} and $\beta_1 = 0.5$
- Loss: Binary Cross Entropy (BCE)

To stabilize training, label smoothing and label noise were added:

- Real labels: $1 + \mathcal{U}(-0.25, 0.25)$
- Fake labels: $0 + \mathcal{U}(0, 0.25)$

1.5 Results

1.5.1 Generated Images Over Epochs

The model was configured to generate and display images at every 10 epochs starting from the 1st. The following figure shows qualitative results at epochs 1, 11, and 21:

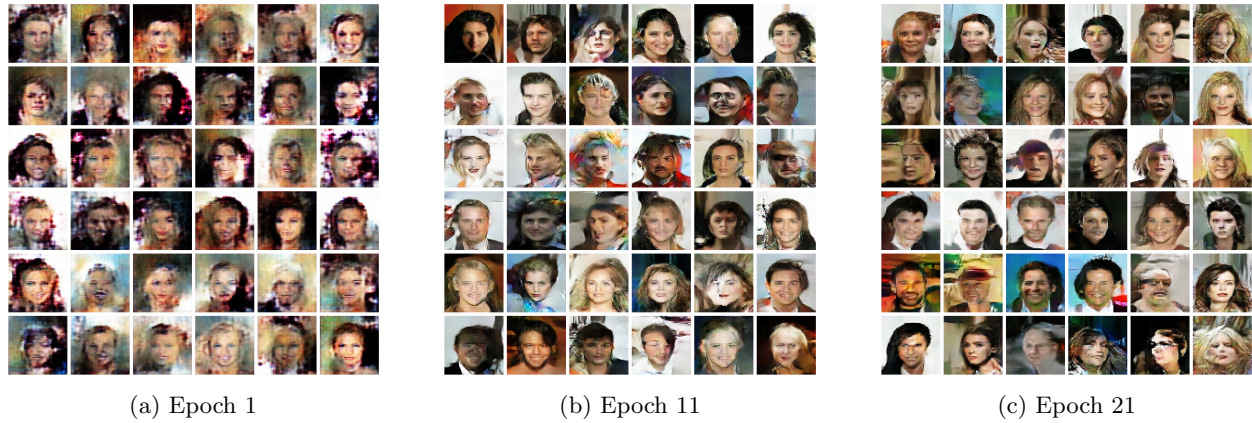


Figure 1: Sample generated face images at different training epochs.

1.5.2 Loss Curves

The following plot shows how the discriminator and generator losses evolved over the course of training:

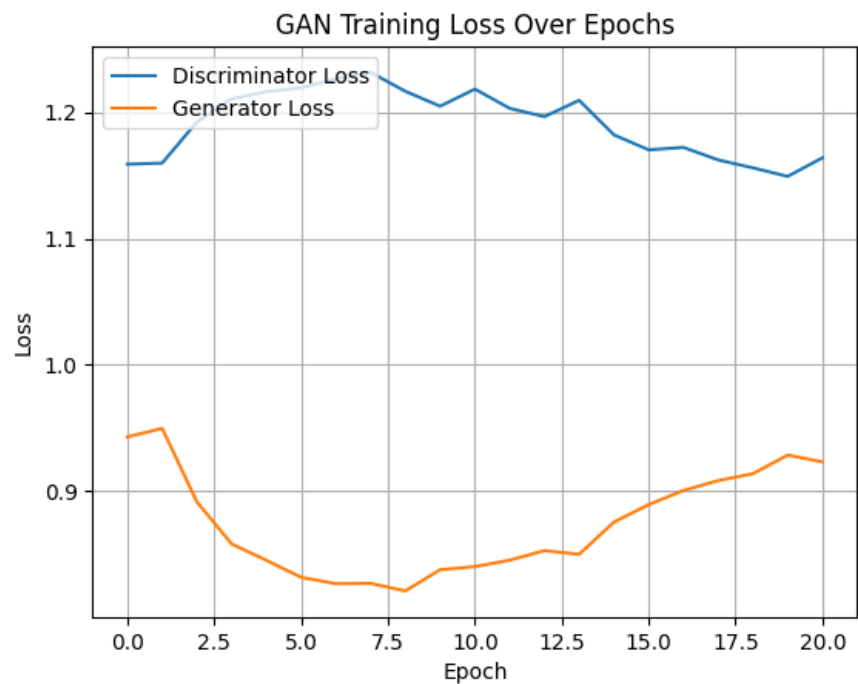


Figure 2: GAN loss curves over 21 epochs.

1.6 Discussion

Initially (Epoch 1), the generator produces low-quality outputs resembling noise. However, as training progresses, images become sharper and more structured. By Epoch 21, generated faces exhibit recognizable facial features such as eyes, mouth, and hair outlines.

The generator and discriminator losses fluctuated but remained within a stable range, indicating that the adversarial training dynamic was well maintained. The use of Batch Normalization and label smoothing likely contributed to training stability and improved convergence.

Despite not reaching photorealism, the DCGAN was able to learn key facial features from the dataset. Further improvement could involve training for more epochs or experimenting with deeper generator architectures and perceptual loss functions.

2 Long-Haired Face Generation using Conditional GAN

2.1 Objective

The aim of this task was to generate synthetic face images conditioned on the presence of long hair using a Conditional Generative Adversarial Network (cGAN). Since the CelebA dataset does not include a “long hair” attribute explicitly, a proxy strategy was employed using gender information.

2.2 Dataset and Label Proxy Setup

We used the CelebA dataset along with the provided attribute labels. However, the dataset does not contain a direct attribute for hair length. To approximate the “long hair” condition:

- The `Male` attribute was used as a binary proxy.
- If `Male` = 0 (i.e., labeled female), the image was assumed to represent a person with long hair.
- If `Male` = 1, the image was assumed to represent short hair.

This binary label was extracted and normalized, then used as a conditional vector input for both the generator and discriminator networks.

2.3 Model Architecture

Generator:

- Input: A noise vector $z \in \mathbb{R}^{100}$ and a binary label vector (female: 1, male: 0).
- Label vector was spatially repeated and concatenated with convolutional layers via Conditional Batch Normalization (CBN).
- Four upsampling (transpose convolution) layers with BatchNorm and ReLU.
- Final output layer uses `tanh` activation to output RGB image.

Discriminator:

- Input: An image and a condition label.
- Label is processed into a spatial tensor and concatenated with the image along the channel dimension.
- Four convolutional layers with LeakyReLU and spectral normalization.
- Final sigmoid layer outputs real/fake probability.

2.4 Training Setup

- Binary Cross Entropy (BCE) loss used for both Generator and Discriminator.
- Optimizer: Adam with learning rate 2×10^{-4} , $\beta_1 = 0.5$.
- Real images conditioned on “long hair” (i.e., Female) were assigned label 1, generated images label 0.
- Training duration: 2 epochs with batch size of 64.
- Generated samples saved every 200 batches for visualization.

2.5 Results



Figure 3: Images generated by the Conditional GAN conditioned on proxy “long hair” label.

2.6 Discussion

- Even with limited training (2 epochs), the model learned to differentiate between facial features loosely associated with the gender-based hair length proxy.
- Generated “long-haired” samples showed more delicate features and occasionally longer hairstyles, indicating the model picked up the correlation between gender and hair length.
- The use of Conditional Batch Normalization (CBN) allowed the generator to inject label-specific context during image synthesis.
- Visual logging helped observe attribute consistency over training iterations.

2.7 Limitations and Improvements

- Using gender as a proxy for hair length introduces bias and does not generalize well.
- A better approach would involve custom annotation of hair length or training on datasets with explicit “hair length” labels.
- Increasing the number of epochs and dataset diversity would improve sharpness and attribute realism.

3 Generating Painted Form of Faces Using CycleGAN

3.1 Objective

The goal of this experiment is to transform facial images into a painting-like style using the Cycle-Consistent Generative Adversarial Network (CycleGAN). This is achieved by translating images from the CelebA dataset (realistic faces) into the artistic domain of the Monet paintings from the Monet2Photo dataset.

3.2 Datasets Used

- **CelebA Dataset:** Contains over 200,000 celebrity face images. We used 10,000 of these images as the input domain A .
- **Monet2Photo Dataset:** This dataset includes artistic paintings by Monet. We used 10,000 Monet images to represent the target domain B .

3.3 Model Architecture

The CycleGAN model consists of:

- Two Generators: G_{AB} (CelebA $\rightarrow$ Monet) and G_{BA} (Monet $\rightarrow$ CelebA)
- Two Discriminators: D_A and D_B , each distinguishing between real and generated images in their respective domains.

Each generator follows an encoder-ResNet-decoder architecture with 6 residual blocks. The discriminators use PatchGANs with progressively increasing convolutional channels.

3.4 Training Details

- **Image Size:** 128x128 RGB images
- **Loss Functions:**
 - Adversarial Loss (MSE)
 - Cycle Consistency Loss (L1): Encourages $G_{BA}(G_{AB}(A)) \approx A$ and vice versa.
- **Optimizers:** Adam with learning rate 2×10^{-4} , $\beta_1 = 0.5$, $\beta_2 = 0.999$
- **Epochs:** 20

3.5 Results

By epoch 20, the generator G_{AB} successfully learned to translate realistic faces into Monet-style painted images while preserving key facial features. The discriminator losses stabilized, indicating that the model reached a balanced adversarial training stage.



Figure 4: CelebA to Monet-Style Translation using CycleGAN at Epoch 20. Top: Real CelebA Images. Bottom: Monet-Style Generated Outputs.

3.6 Analysis

CycleGAN was effective in unpaired image-to-image translation between human face photographs and artistic portraits. The model learned domain-invariant structures (e.g., facial layout) while mapping textures and colors from the Monet domain. This success was made possible through cycle consistency, allowing learning from unaligned datasets without explicit pixel-wise supervision.

3.7 Conclusion

CycleGAN successfully learned the transformation from real faces to painted portraits using unpaired datasets. This technique can be extended to style transfer tasks in other domains, such as sketch-to-photo, summer-to-winter, and day-to-night transformations.

4 Explanation for Using BCE Loss Instead of MiniMax Loss in TensorFlow GAN Implementations

4.1 Background

In the original Generative Adversarial Network (GAN) formulation proposed by Goodfellow et al. (2014), the objective is defined as a minimax game between the generator G and the discriminator D :

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] + \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

The discriminator aims to maximize this function by correctly distinguishing real data x from fake data $G(z)$, while the generator tries to minimize it by generating realistic samples that fool the discriminator.

4.2 Issue with MiniMax Loss for Generator

The original generator loss:

$$\mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

tends to saturate when $D(G(z))$ is close to 0, particularly during the early stages of training. This leads to vanishing gradients, making it difficult for the generator to learn.

4.3 Non-Saturating Generator Loss (BCE Loss)

To address the vanishing gradient problem, the authors proposed an alternative loss for the generator:

$$\min_G -\mathbb{E}_{z \sim p_z} [\log(D(G(z)))]$$

This formulation provides stronger gradients and improves the training dynamics. It is essentially the Binary Cross-Entropy (BCE) loss used in binary classification.

4.4 Binary Cross-Entropy (BCE) Formulation

The BCE loss function is:

$$\text{BCE}(y, \hat{y}) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

In GANs:

- For real samples ($x \sim p_{\text{data}}$), the label $y = 1$
- For fake samples ($x = G(z)$), the label $y = 0$

Discriminator Loss:

$$L_D = -\mathbb{E}_{x \sim p_{\text{data}}} [\log D(x)] - \mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$$

Generator Loss:

- **MiniMax Loss:** $\mathbb{E}_{z \sim p_z} [\log(1 - D(G(z)))]$
- **BCE (Non-saturating) Loss:** $-\mathbb{E}_{z \sim p_z} [\log(D(G(z)))]$

4.5 Why TensorFlow Uses BCE Loss

TensorFlow and other deep learning frameworks prefer the non-saturating BCE loss for the generator because:

- It avoids vanishing gradients
- It improves numerical stability (avoids $\log(0)$)
- It accelerates convergence during training
- It results in empirically better performance

4.6 Conclusion

While the MiniMax loss reflects the theoretical formulation of GANs, the BCE loss improves practical training. TensorFlow adopts this non-saturating BCE loss as a default in most of its GAN tutorials and implementations.

Table 1: Comparison Between MiniMax and BCE Generator Losses

Loss Type	Equation	Used For	Problem Solved
MiniMax Loss	$\log(1 - D(G(z)))$	Generator	Suffers from vanishing gradients
BCE Loss	$-\log(D(G(z)))$	Generator	Provides stronger gradients and stable training

Code Link:

Here's the github link of the implemented code: <https://github.com/mdsajjadhossain25/CSE4261>